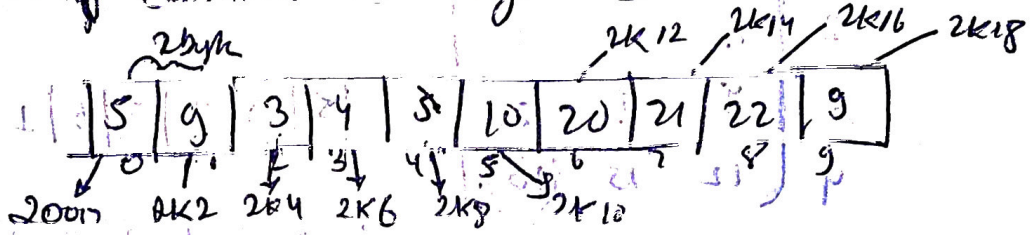
Arrays

Base Address :- ~~from where~~ the element address at that point

int a[10] = { 5, 9, 3, 4, 5, 10, 20, 21, 22, 9 }

Base address of array is 2000 and size of element is Byte 2



location → LOC of A[5] = 2010

Golden rule = no of element skip X byte + Base address

No. of elemen = U.B − L.B + 1

2D Array

int A[4][3] ~~in~~ C language array start from 0 slot it is (5x4) matrix

In memory there is no 2D array there is only 1D all things are stored in 1D type of Array.

	1	2	3
1	5	9	3
2	8	6	1
3	4	7	2
4	12	15	10

Row major fashion

|5|9|3|8|6|1|4|7|2|12|15|10|

|5|8|4|12|9|6|7|15|3|1|2|10|

↑ Column major fashion

Row major LOC A[4][3]

3 rows we skip

starting from 1 so 1+03 we skip then we reach to 4 thus 3-1+1 = (3)

for 1st column 1 to 2 we skip so 2-1+1 = (2)

3 rows each row 3 element so 9,

after 9 element we reach to 4th Row now we will skip this (2) to reach the A[4][3]

So total 11 (9+2) ... need to skip

Column major

Inc A[3][2]

↓ i - 1 + 1 = 1 Column skip

1 to 2

2 - 1 + 1 ⇒ 2 ⇒ 2 element skip

∴ 1 colm contain 4 element so (4+2) = 6 total

3D Array

assuming first element is A[1][1][1]

a[10][20][30] → if C language then A[0][0][0]

↓ 10 2D Array of 20×30

LOC A[5][6][7]

↓ Row major

4-1+1 | 6-1+1

6420 Array | 6 - 5 - 1 + 1 = 5 = (5)

will have skipped = (5 × 30)

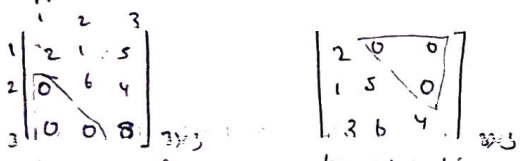
total element is 4×20×30

total element skip = 4×20×30 + 5×30 + 6

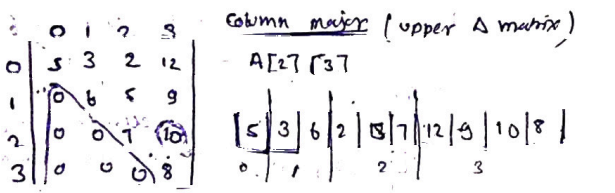
Same as column major.

(4×20×30 + 5×30 + 6) size + Base address

Upper and lower Δ matrix



Upper Δ matrix Lower Δ matrix



Column major (upper Δ matrix)

A[2][3]

A[2][3]

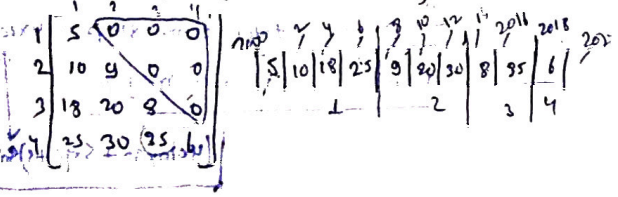
0 to 2 ⇒ 2−0+1 = 3 column skip

3(3+1)/2 elements

1+0+1 = 2 element skip

so total ⇒ (3(3+1)/2 + 2)

Column major (Lower Δ matrix)



A[x][y] → [(upperbound-1) - lowerbound+1]

5/__/__

A[x][3] → 3 skip the column

2-1+1 = 2

3-1+1 = 3 → skip the element

$\frac{2(2+1)}{2}$

*** first we need to search at the end position then we will start from there so

total element we skip to reach at last is,

$\frac{4(4+1)}{2} - 10$

A[4][3] → element

upperBound - lowerBound

4-3+1 => 1 => $\frac{2\times(2+1)}{2} = 3$

3-3+1 = 1 element

i.e. Row - colum (4-3) = 1

(10 - 3 + 1) = (8) total 8 elemt skipped

And think for Row major (upper Δ matrix)

A[3][4]

4-3+1 = (2) $\frac{2\times(2+1)}{2}$ (3)

$\frac{2\times(4+1)}{2}$ = (10) and element => 4-3 = (1)

Total elemt skipped to Richest last

(10 - 3 + 1)

7+1 = (8)

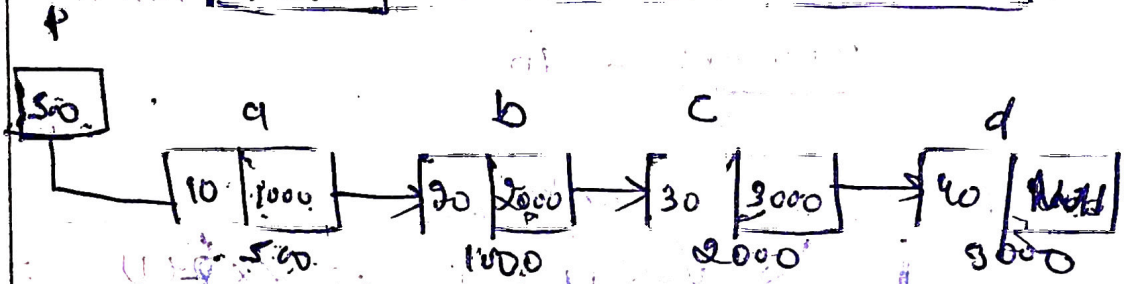
Linked List

multiple list which are linked

int *p; p is pointer to int variable

float *p p is pointer to float variable

| Value | Pointer to next variable |



printf((*p).val) (output is 10)

printf ((*p).next) (output is 2000)

a ka address 500 hai / b ka address 1000 hai

p = Address of A

p = p → next // 1000

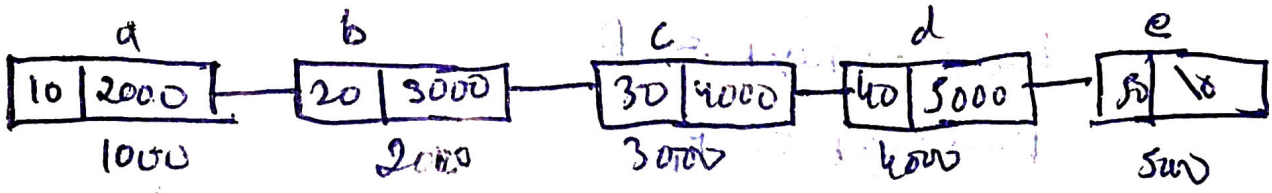
p = p → next // 2000

p = p → next // 3000

p = p → next // Null

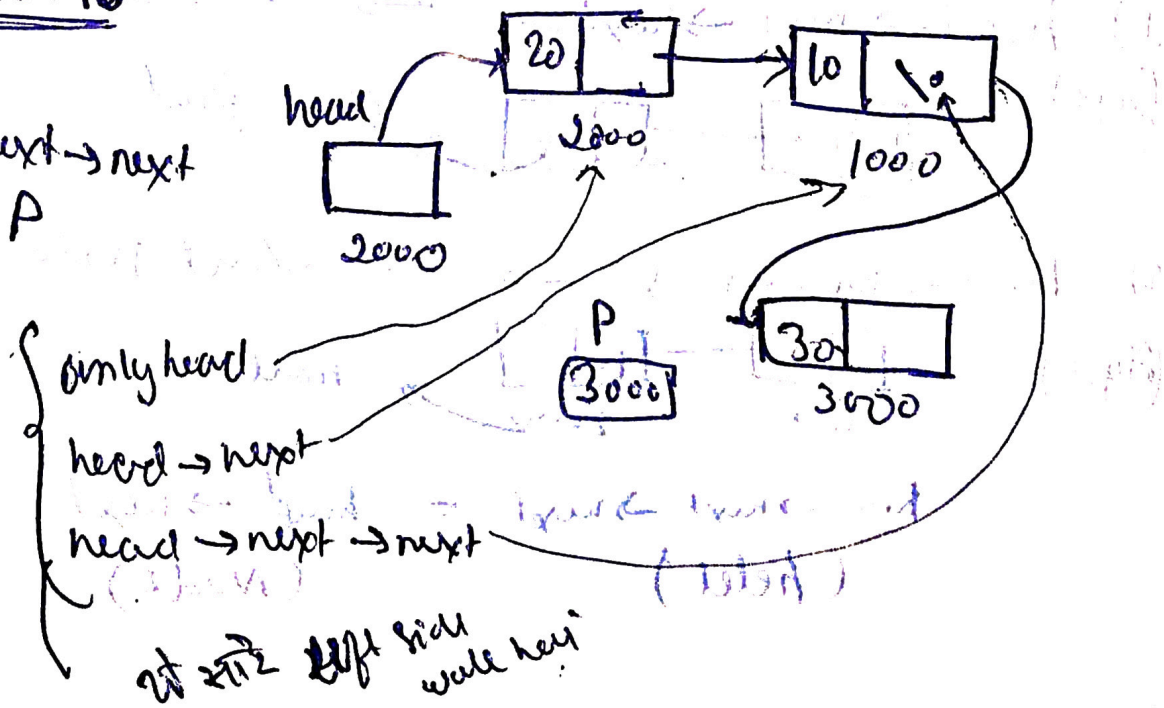
```
List () {
Struct node a = {10, NULL};
struct node b = {20, NULL};
struct node c = {30, NULL};
struct node d = {40, NULL};
Struct node e = {50, NULL};
  a.next = &b;
  b.next = &c;
  c.next = &d
  d.next = &e
}
```

After writing this Code a linked-List will be made

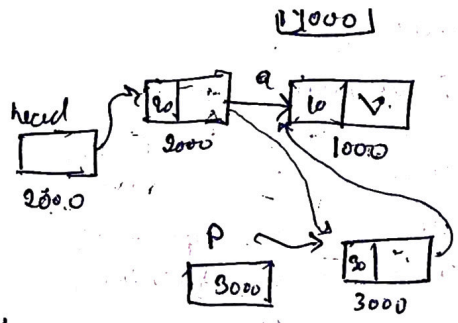


Add P bet 10

head→next→next = P



at the starting



insert p at q

p->val = 30;

If we write :

head->next = p;

But But if we do like this then we will lost access for 10 node

So make a new pointer

Struct node * p1 = NULL

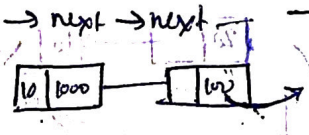
p1 = head->next

head->next = p

p->next = p1

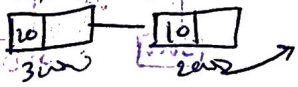
(1) head->next->next → Talk about field

(left)



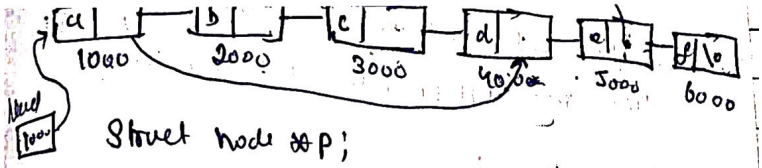
(2) head->next->next → Talk about node

(Right)



head->next->next = head->next

(field) (Node)

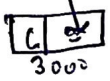


Struct node *p;

p = head → next → next ;

// points at 3000

p → next → next → next = p;



head → next = p → next

p f (head → next → next → next → next → val);

1000 → 4000 → 5000 → 3000 → 4000 → d

Key

p → next → next → next → This will talk about field

3000 → 4000 → 5000 → field.

If this is present in left side.

head → next → next = p → next → next → next

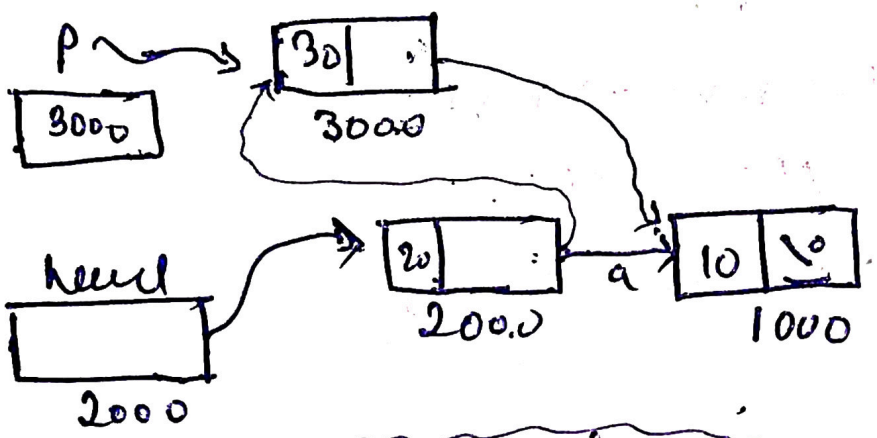
talk about node

talk about node

last "next"

talk about field

If we dont want to make a different pointer then means efficient way to solve this




```
P -> next = head -> next
head -> next = P;
```

If we try to write

```
head -> next = P;
P -> next = head -> next
```

→ means .. node [10|] is last

If list is empty then Segmentation fault will occur

If node has two pointers then only we can change the pointers otherwise we will loose the access of after that pointer __/__/___

p → val = 10

p → next = 1000

p = p → next · // 2000 but we will loose the access to move back so we will make a new pointer and old one we will move that

struct node *p1

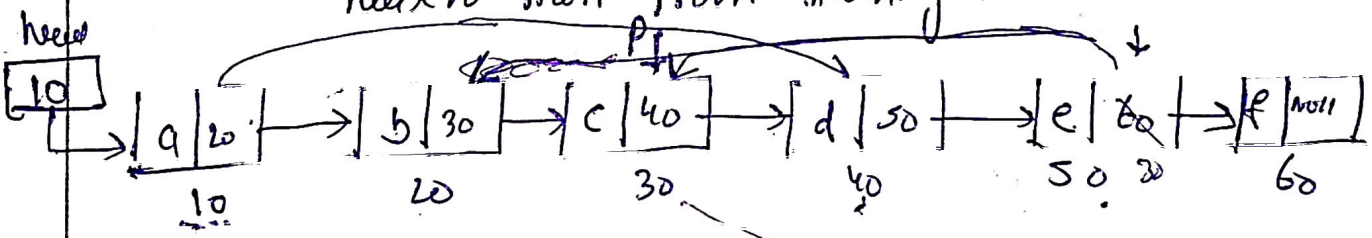
p1 = p → next

p1 = p1 → next

Gate 09 **Note:** to access previous node in Single Linked list we need O(n) time in worst case

Reason B/c directly we can't move backward we need to start from starting.

Q

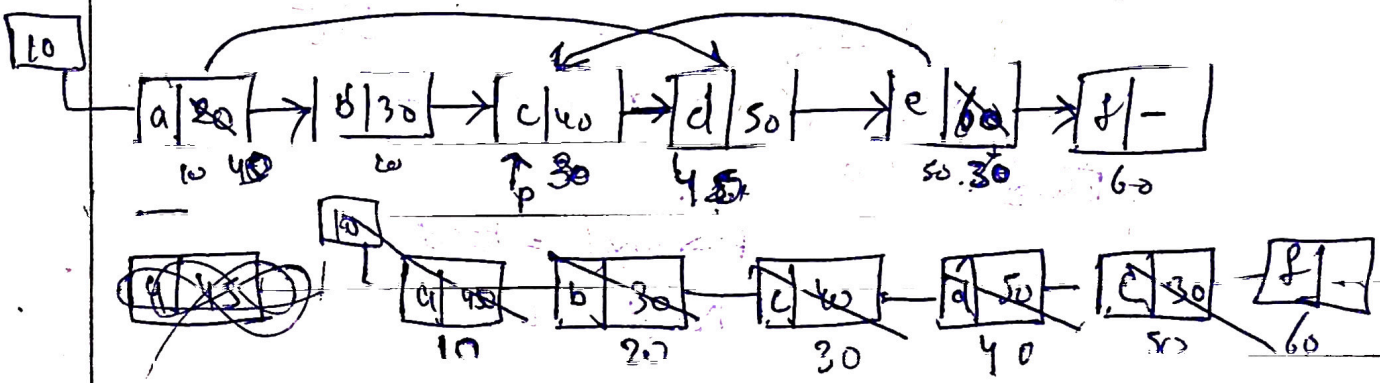


struct node *p;

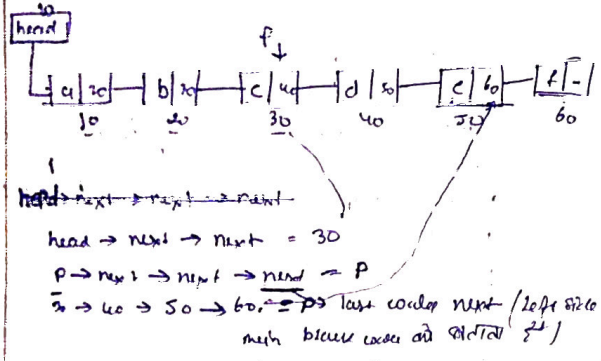
p = head → next → next; // 30

p → next → next → next = p // 60 = 30

head → next = p → next // 20 = 40



Important Rules

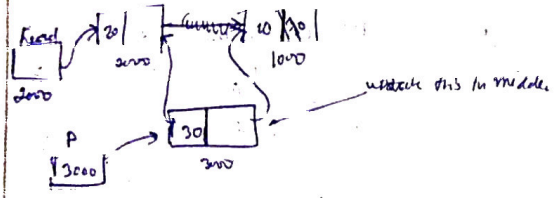


head → next → next = 30

p → next → next → next = p

30 → 40 → 50 → 60 → p last cocha next (left side mein bacche cases aur right ke)

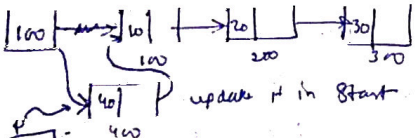
#



p → next = head next ;

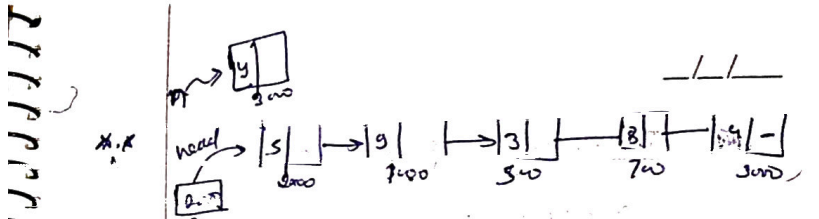
head → next = p ;

#



p → next = head

head = p



(Traverse the Linked List)

```
Traverse(Struct node *p) {
  while (p -> next != NULL)
    pf (p -> val);
    p = p -> next
}
```

This will not Print the last value of the node.

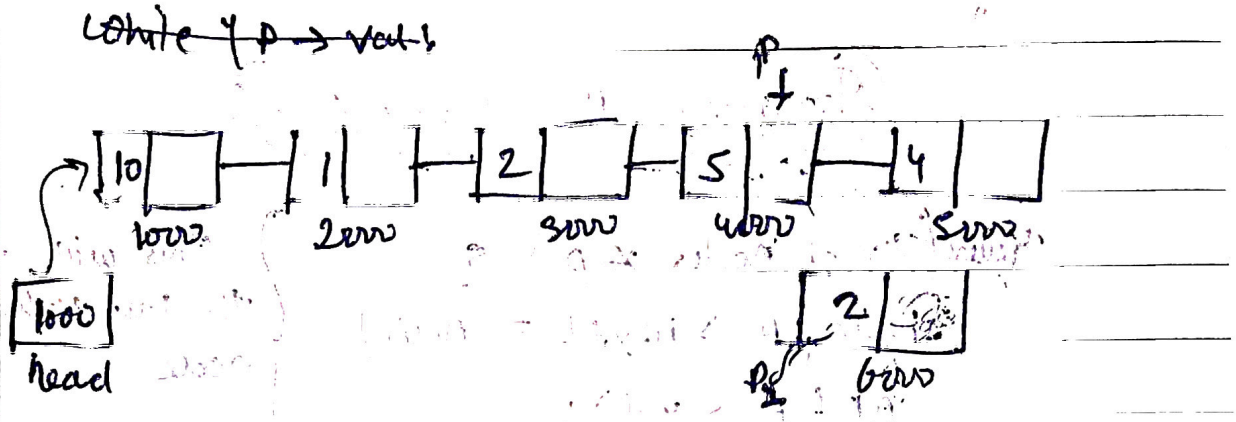
So we need this.

```
if (p == null)
  pf ("List is empty")
else
Traverse(Struct node *p) {
  while (p != null) {
    pf (p -> val);
    p = p -> next
  }
```

Insertion

```
Struct node *n = (Struct node *) malloc (size of (Struc
n -> val = y
n -> next = head
head = n
```

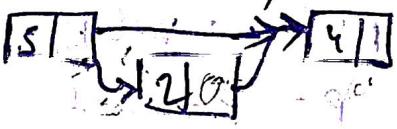
Insertion after a given node:




```
Insert (struct node *p, int x, int y);
while (p->val != 5)
    p = p->next;
```

struct node *p1 = (struct node*) malloc (sizeof (struct no

// we need connect



SO

```
p1->next = p->next
p->next = p1
```

Note:

Always check the list is empty or not

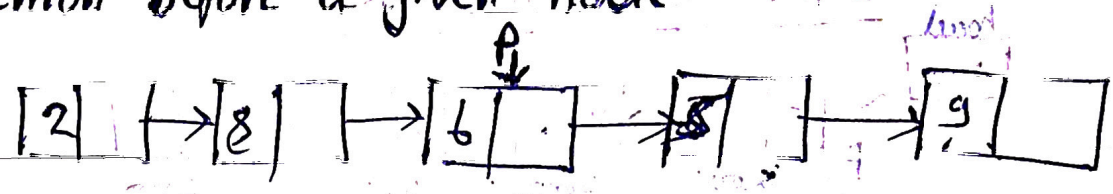
for this we will write

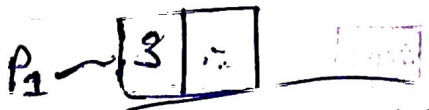
```
while( p1 != NULL && p->val != 5)
```

Note:

If multiple values are present then the very first value will be updated.

Insertion before a given node

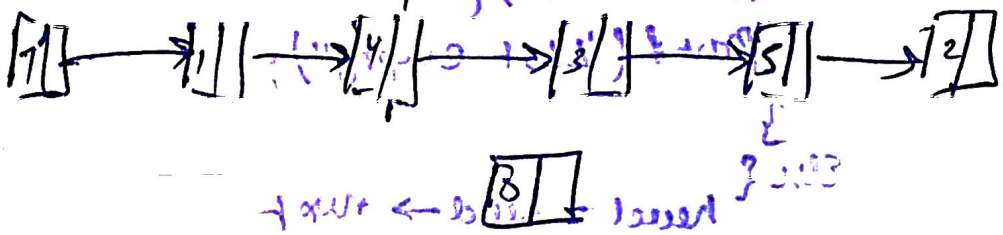





```
while (p -> next -> val != 5 && p -> next != null)
```
first check this

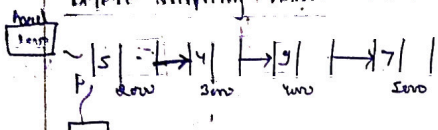
```
    p = p -> next
if (p -> next != null)
struct node *p1 = (struct node *) malloc (sizeof (struct node))
    if (p1 == null) {
        printf ("not found")
    }
else  p1 -> val = 3
      p1 -> next = p -> next
      p -> next = p1
```

If we want two node before




```
p = p -> next -> next
while (p -> val != n && p -> next != null) {
    p1 = p1 -> next
    p = p -> next
}
```

Delete starting node



free (head); } element of this because close to this the pointer (p) that he is pointing will
free (p); } be free not the pointer. ∞

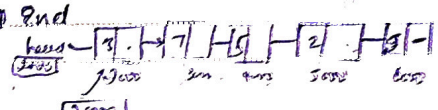
free(p); 4 byte of memory from 2000 to 2003 is given back to O.S (operating system)

⇒ pointer is pointing to some memory only empty the O.S has given to memory to others process

⇒ Indirectly we can say that head and p are pointing to that location which does not exist according to user view.

if(p == null) {
printf("List empty");
}
else {
head = head -> next
free (p);
p = null;
}

Delete at End



```
while (p->next->next != null)
    p = p->next
free (p->next);
p->next = null;
```

This code is not applied for single node.

Before this we will put

```
if (p == null)
    pf ("LLs empty");
else if (p->next == null)
    free (p)
    p = null
```

Deletion of node after node

```
if (p == null || p->next == null)
    pf ("node not found")
else
while (p->val != s && p->next != null)
    p = p->next
if (p->val == s)
    p->next = p->next->next
```

18/__/____

The given program couldn't check that the node is at last position or not for this we will do little modification

We will do (if (p→next != null))

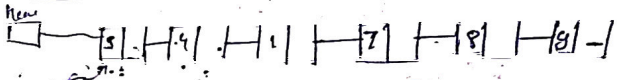
Full code

```
if (p == null || p→next = null) {
   pf ("node not found")
}
else {
  while (p→value != s && p→next != null) {
        p = p→next
  if (p→next != null) {
     p→next = p→next→next
  }
  else { printf("node not found")
     }
}
```

Mindit

Get code from him check out the new node to assign with node of the deleted node. When we will not get node after the node then it will create problem.

Deletion before the node




```
Delet (struct * p)
if (p==null or p->next==null or p->val==5)
    Print ("Node not found")
}
else if (p->next->val==5){
    head = head->next
    free (p)
    p = null
}
else if (p->next->next==null){
    print ("Node not found")
}
else{
    while (p->next->next->val!=5 && p->next->next->next!=null)
    p = p->next
    if (p->next->next->val==5){
        p->next = p->next->next
        struct node *p1 = p1->next
        p->next = p1->next
        free (p1)
    }
    else{
        print ("Node not found")
    }
```

2 Node ka Checking

2 Node ke baad agar value 5 nahi hai

5 Node Checking

20/__/____

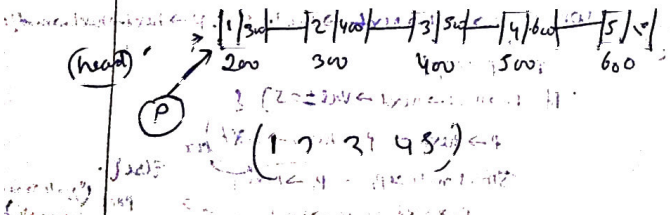
if (p) कहां पर question में लिखा है मतलब (p != NULL)

(i) (List->next == Null)
!(list->next)

(ii) while(q)
↳ similar while(q != Null)

Q7A Traverse the linked list by Recursion

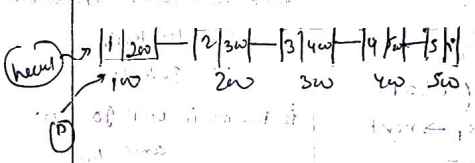
```
Void main ;{
Traverse ( head );
Traverse ( Struct node *p ) {
  if ( p ) {      // if(p != 0)
    print ( p -> val );
    Traverse ( p -> next );
  }
}
```

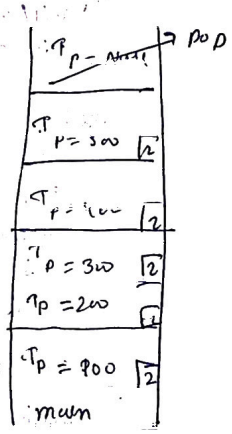


~~Reverse the linked list~~

Traverse the Linked list in opposite

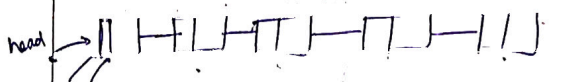
```
void main ()
{
    traverse (head);
}

traverse (struct node *p)
{
    if (p)   // if (p != 0)
    {
        traverse (p -> next);
        printf (p -> val);
    }
}
```





22/__/____

How to find mid of linked list




```
struct node *mid(struct node *s)
{
    if(s==0)
        return NULL;
    else
    {
        s1 = s;
        c=0;
        while(s!=0)
        {
            s = s->next;
            c++;
        }
        c = c/2;
        while(c!=1)
        {
            s1 = s1->next;
            c--;
        }
        return s1;
    }
```

```
(struct node *) mid(struct node *s)
{
    s1 = s;
    while(s1 && s1->next!=0)
    {
        s1 = s1->next->next;
        s = s->next;
    }
    return s;
}
```

suppose a car is travelling at 100 km/h at 50 km/h

in next hr it will go 200 and 100

then next 300 and 150

means always half thus we implemented this here

T.C = O(n)

23/__/____

Binary Search on Linked List

```
(struct node *) BS (struct node *p, int n)
    if (p == NULL)
      printf ("Linked List is empty")
          return p
    else if (p -> next == NULL)
        if (p -> Val == n)
            return p
      else
            print ("Node not found")
                return null;
             3
else {
    struct node *S
    S = mid(p)      // mid function pehle bnaya hai
   if (S -> val == n){
        Printf ("node found").
        return S;
   else if (S -> val > n)
      S -> next = Null
      BS (P, n)
    else
        BS (S -> next, n);
   }
```

24 /_ /___

→ Binary search can be applied on sorted array as well as Linked List

→ Only difference is mid finding in array A(1) and Linked List A(n)

→ Hence B.S is not efficient on List

→ LS is better than BS for LLS

∵ BC = A(1) { Best case (BC)
WC = O(n) { Worst case (WC)

But Binary Search need O(n)

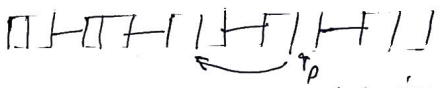
Circular Linked List

We can store 1st node address in place of storing null at the last position in Single Linked List

Advantage

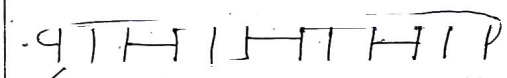
Main use : utilizes last node next part & can reach to 2nd node if we are at 3rd (middle) node in A(n)

Single Linked List



Circular Linked List

Insert node with data x at end of CLL



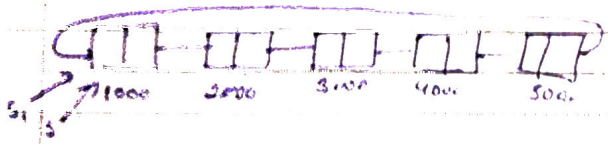
```
struct node *p
p=(struct node *) malloc sizeof (struct node);
if (p==NULL)
   printf ("Memory not available");
   return;
} else
     p->data = x
     p->next = NULL
if (s == NULL)
     s = p;
     s->next = s
     return s;
```

} making a node that we have to insert

```
else {
    S1 = S
    while (S1->next != S) {
        S1 = S1->next
    }
    P->next = S;
    S->next = P;
}
```

TC O(n)

Delete 1st node in Circular Linked list




```
if (S == NULL)
    return S;
S1 = S
while (S1->next != S)
    S1 = S1->next;
P1->next = S->next
free(S)
S = S1->next
```

The previous code will not be executed for single node

for single node we will add small thing

```
if (s->next = s) {
    free(s)
    s = NULL;
}
```

5) when we can say it is 1st node of Double linked list

```
if (s->prev == NULL)
    pf("1st node")
```

6) when we can say it is last node of DLL

```
if (s->next == NULL)
    pf("1st node")
```

6) How to get address of 1st last node of DLL

```
while (s->next != NULL)
    s = s->next;
    s = s->prev;
```

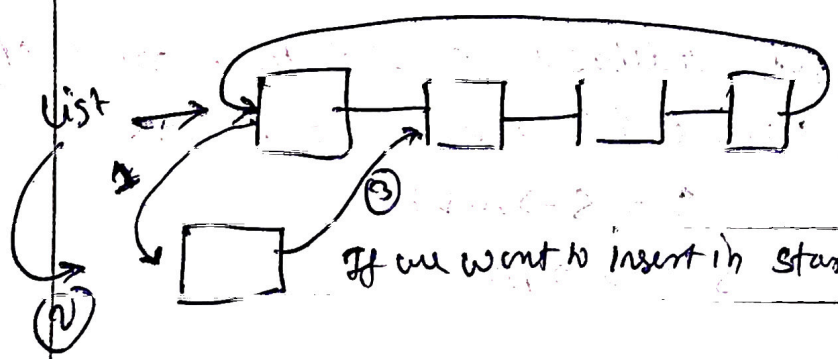
Q7) How to print elements in reverse order in DLL

```
while ( s -> next != NULL) {
        s = s -> next
    }
    while ( s -> previous != NULL)
        printf ("%d", s -> value);
        s = s -> prev;
    }
free(new)
```

Gate 1987

In a circular linked list organization, insertion of a record involves modification of

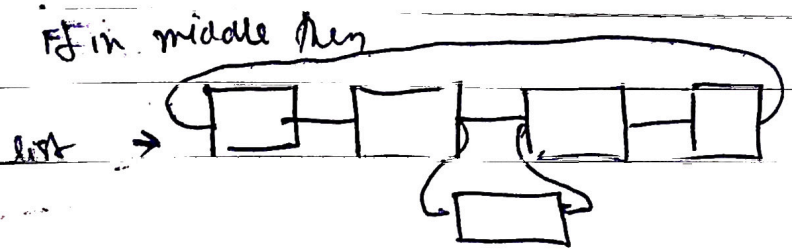
1) Two pointers ✓
2) one pointer ✗
3) No pointer ✗
4) multiple pointers ✗ → this can be true if we insert at starting



list will point to 1

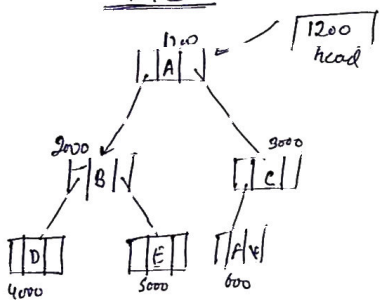
If we want to insert in starting ③ Changes

If in middle then



29/__/____

Tree



> Tree is acyclic connected graph
> Any tree having atmost 2 children is called binary tree
> Its like Doubly linked list but difference is that in DLL A point to B and vice versa and in Binary tree if A point to B then B never point to A [To avoid cycle]

linked list given means → pointer to first node given
Tree given means → pointer to root node given

(Here we will call the pointer as root)

Q) When can we say binary tree is empty

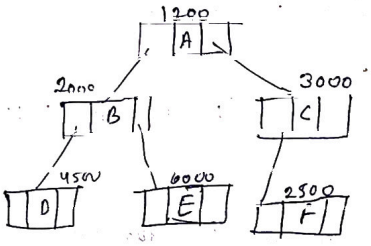
```
if (root == Null)
```

Q) When can we say binary tree has only one node

root → lc == Null and root → rc == Null

(We can also say NIL in application is BT)

Q) How to count location in Binary tree.?



Type 1

```
int LN (struct node *p)
  if (p == Null)
     return 0
  else if (p->left == Null && p->right == Null)
     return 1
  else
     L = LN (p->Left)
     R = LN (p->Right)
     T = L+R
     return T
}
```

n

Type 2

Instead of n we can also write

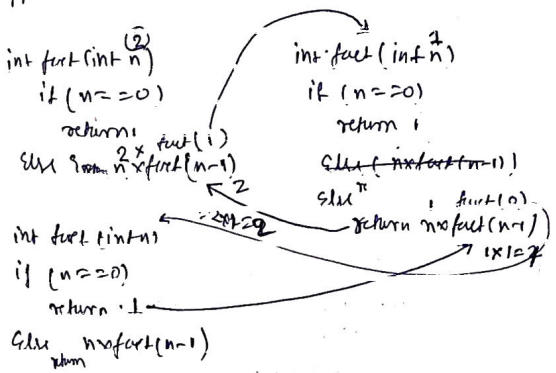
```
else
     return (LN (p->Left) + LN(p->right))
}
```

Extra VIP

How actually the function call works in recursive call

```
int fact (int n)
  if (n==0)
     return 1
  else  return n*fact(n-1)
```

Suppose I have taken $n=2$



So overall 2 will be multiplied

Time complexity is the height n nodes and for each node we call a function once

$\Rightarrow O(n)$

We can also write in

$$T(n) = 2T(n/2) + c$$

Analysis

Time complexity

B.C = Ω(n)
A.C = Θ(n)
W.C = O(n)

Space complexity

Ip + extra

B.C = n + log(n)
A.C = n + log(n)
W.C = n + log(n)
↓
O(n)

How to count Num leaf Node.

```
int NL(struct node *P)
    if(p == NULL)
        return 0;
    else if(p->left == NULL && p->right == NULL)
        return 1
    else
        return (NL(p->left) + NL(p->Right));
```

How to count Total No of node.

```
int TN(struct node *P)
    if (p == NULL)
        return 0;
    else if(p->left == NULL && p->right == NULL)
        return 1
    else
        return(1 + TN(p->left) + TN(p->Right));
```

VIII If we want InXLN + SXTotalNodes

```
int Node (Struct node *p)
    if (p == NULL)
        return 0;
    else if (p->left == NULL && p->right == NULL)
        return 10;
    else
        return (5 + Node(p->left) + Node(p->right));
```

VIIA Find the (internal nodes - Leaf nodes)

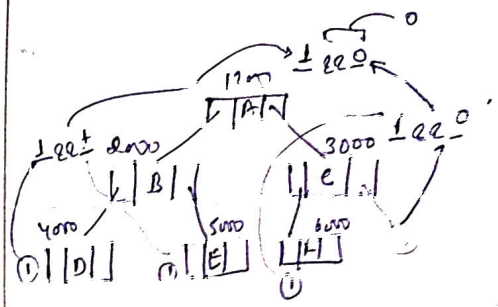
```
int Node (Struct node *p)
    if (p == NULL)
        return 0;
    else if (p->left == NULL && p->right == NULL)
        return -1;
    else
        return (1 + Node(p->left) + Node(p->right));
```

How to find the height

```
int H(struct node *p)
  if(p==NULL)
     return 0
  else if(p->left==NULL && p->right==NULL)
     return 0;
  else
     return (1 + max(H(p->left) + H(p->right)))

int Max(int a, int b)
     if(a > b)
        return a
     else
        return b
```

How to verify given binary tree is strict or not
[every node has either 0 child or 2 child]

```
int SBT(struct node *p)
  if(p==NULL)
     return 1;
  if(p->left==NULL && p->right==NULL)
     return 1
  if(p->left!=NULL && p->right!=NULL)
     return (SBT(p->left) && SBT(p->right))
     return 0;
}
```



TC $T(n) = A(n)$ (Space complexity = $O(n)$)

Write a code when two binary tree are equal

```
Equal(P1, P2)
    if (P1 == NULL && P2 == NULL)
        return 1;
    if (P1 == NULL && P2 != NULL || P1 != NULL && P2 == NULL)
        return 0
    if (P1->Val == P1->Val)
    return (P1->left, P1->left) &&
        return Equal(P1->left, P2->left) &&
                Equal(P1->right, P2->right)

Else
    return 0
```

Time Complexity
worst case $A(n)$
Best case $O(1)$

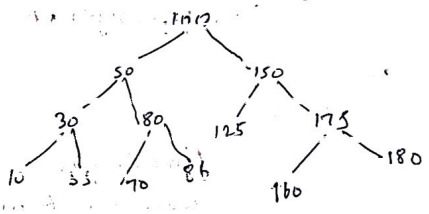
Making the mirror image of Tree

```
mirror (struct node *p):
    if (p == NULL)
        return p
    else if ( p -> left == NULL && p -> right == NULL)
        return p
    else {
        t = p -> leftchild rightchild
        p -> rightchild = (p -> leftchild) mirror
        p -> leftchild = mirroring(t)
    return p
```

Time complexity = O(n)

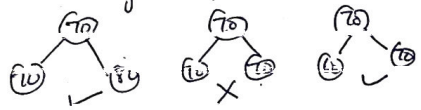
Binary Search Tree

Binary Search and Binary tree are different



=) Left side small and right side big

Suppose if one side (left side) is ≤ (less than or equal to) then don't apply the equal to function on right side



FTS smallest element in Binary Search Tree
⇒ left most element

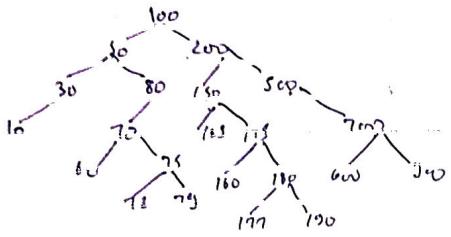
```
while (temp->left != NULL):
    temp = temp->left
}
return temp;
```

Check many times

Time Complexity

B.C = O(log(n))
W.C = O(n)
A.C = O(log(n))

Searching An element



In ordered manner you will search 177

100 → 200 → 150 → 175 → 180 → 177

Time complexity :-

B.C : O(1)

W.C : O(n)

Avg Case : log(n)

Successful Search

	Min element			S.S			LDS/BS		
	BC	WC	AC	BC	WC	AC	BC	WC	AC
BST	$\Theta(1)$	$\Theta(n)$	$\log(n)$	$\Omega(1)$	$O(n)$	$\log(n)$	$\log(n)$	n	$\log(n)$
BT	$O(n)$	$\Theta(n)$	$\Omega(n)$	$\Omega(1)$	$O(n)$	$O(n)$ / $\log(n)$	$\log(n)$	n	$\log(n)$
Min heap	$\Theta(1)$	$\Theta(1)$	$\Omega(1)$	$\Omega(1)$	$O(n)$	$\Theta(n)$	$\log(n)$	$\log(n)$	$\log(n)$
AVL	$\log(n)$	$\log(n)$	$\log(n)$	$\Omega(1)$	$\log(n)$	$\log(n)$	$\log(n)$	$\log(n)$	$\log(n)$

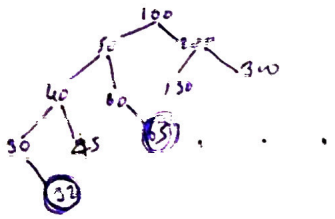
	BC	WC	AC
BST	$\Theta(1)$	$\Theta(n)$	$\Theta(\log(n))$
BT	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Min heap	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
AVL	$\log(n)$	$\log(n)$	$\log(n)$

(1) S.S unsuccessful Search

** while inserting a value in BST it will be implemented as leaf node in middle it will not come

Insertion in BST

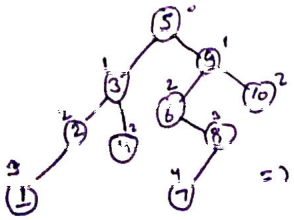
→ Insert 32, 65



How to create BST with n nodes

5, 9, 3, 10, 6, 8, 2, 4, 1, 7

Then will ask how many comparision did you done?



=> 0+1+2+3+4+2+2+1+4

2+3+4

=> 20

Time complexity for ~~Operation~~ creation BST

$BC = \Theta(n \log n)$

$WC = \Theta(n^2)$

$AC = \Theta(n \log n)$

Deletion in Binary Search tree

→ leaf Node

→ 1 child

→ 2 child

(condition for 2child) If we have to delete 2child node then either replace it with successor or predecessor

Predecessor

one time left then right-right most element

Successor

one time right then left-left most element

Successor and predecessor have at most one child if they will never have 2child

Time complexity for deletion of node

Best case = $O(1)$

Worst case = $O(n)$

Avg case = $O(\log(n))$

Time complexity for insertion of node in BST

B.C = $O(\log n)$

W.C = $O(n)$

Time complexity for creation of BST

- Balanced = $n\log n$

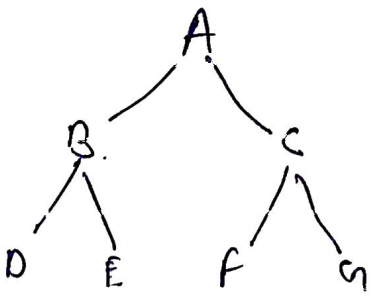
  Unbalanced = n^2

Tree traversal

> Visiting each node of the tree.
> Searching is similar to traversal but searching should be of worst case

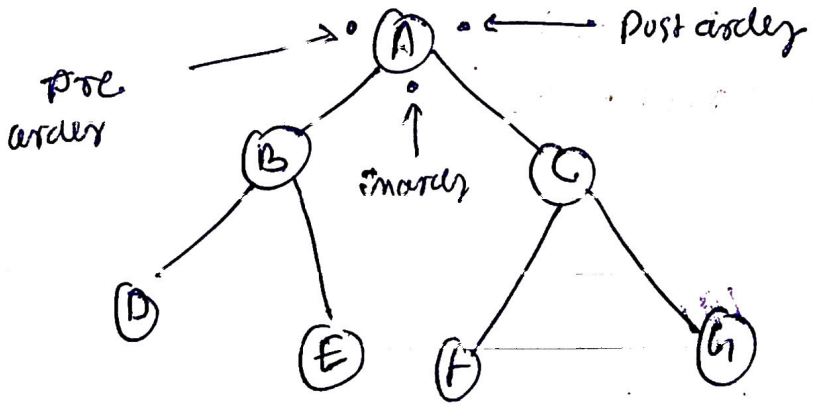
3 way to tree traversal

- preorder → Root → Left → Right
- Inorder → Left → Root → Right
- postorder → Left → Right → Root



pre → A B D E C F G
In → D B E A F C G
post → D E B F G C A

preorder (Root first)
postorder (Root Last)

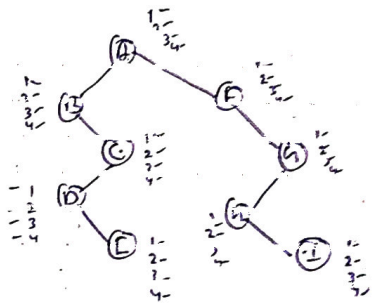


```
preorder(root)
if( root == NULL)
    return root
elseif( root != NULL)
  Printf( Root )
  preorder( root → left)
  preorder (root → Right)
```

Time Complexity	Space Complexity
BC → Ω(n)	BC → O(n)
WC → O(n)	WC → O(n)
AC → Θ(n)	AC → Θ(n)

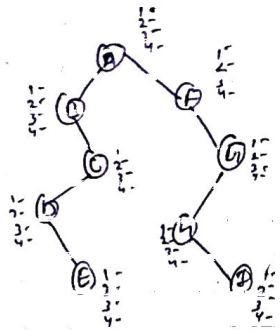
Double order Traversal

```
Do (root) {
  if (root) {
(1)  Pf ( root → val)
(2)  Do ( root → le)
(3)  pf ( root → val)
(4)  Do ( root → rc)
```



A BBC DDEE CA FFG HHH I IG

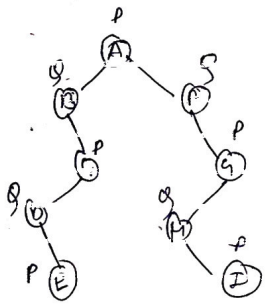
DO (root)
if (root) S
① Do (root → lc)
② Pf (root → val)
③ Do (root → rc)
④ Pf (root → val)



B D F E D C C B A F H I I H G G F A

Indirect Recursion

```
P(root) {
if (root) {
   Pf(root->val)
   Q(root->lc)
   Q(root->rc)
}}

Q(root)
   if(root) {
      P(root->lc)
      Pf(root->val)
      P(root->rc)
```

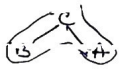


We are starting from P
output : A B C D E F G H I

Note The order will always be in ascending order
Just print the array that will be
in order

VIII If Post and In order are given then how to find the pre ordered.

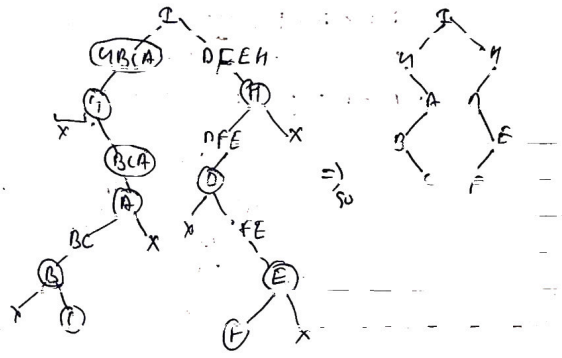
Post : A B C — Root
In : B C A
L R
pre ??



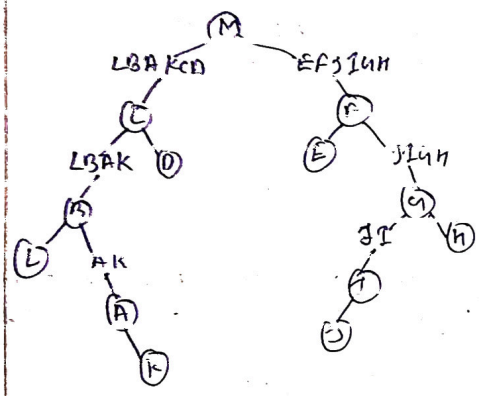
Pre : C B A

Note (Pre) or (Post) so hme Root पता चलता है
(In) so hme left or Right पता चलता है

Pre : I G A B C H D F E
In : G B C A I D F E H
L R

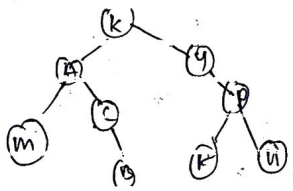


Post : LKARDEFJIHGFM
In : LBAKCDMEFJIGH



Identify pre, In, Post

MBCAFHPYK → Post
KAMCBYPFH → Pre
MABCKYFPH → In



⇒ Two case when to make tree when

Pre and In are given ✓

Post and In are given ✓

→ BST (Binary Search tree)

(Pre: 50 40 30 20 10 32 34 36 42 44 60

In: ??

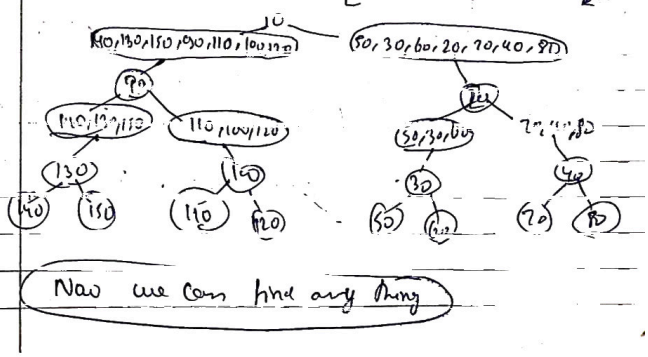
↳ In is always in sorting form

In = 10, 20, 30, 32, 34, 36, 40, 42, 44, 50, 60

After this we can find the post also how we have got pre and In

Min heap given

In: 10, 130, 150, 90, 110, 100, 120, 10, 50, 30, 60, 20, 70, 40, 80



Now we can find any thing

Q How many tree are possible

→ ① Pre and Post given → ≥ 0
② Pre and In given ≤ 1
③ post and In given ≤ 1

Q Binary Tree Construction from pre and In order
or Post and In

BC → $\Theta(n)$
WC → $\Theta(n^2)$ → n nodes + level
AC → $n \log(n)$

Q Pre order Traversal of BST is given find Postorder Traversal of BST algo T.C

Post Order

Pre order is given
Step1 In order (Sort the pre order) → $\Theta(n \log n)$
Step2 Construct unique BST from Pre & In → $\Theta(n)$
Step3 Find the Post from the tree → $\Theta(n)$

So overall $\Theta(n \log n)$

5/__/____

* If pre and In order given then time required to construct the Binary Search tree = $\Theta(n)$ in Best case

Best case

Step 1 Sort In order (Sort the preorder) → $\Theta(n \log n)$

Step 2 Construct the unique tree (BST) from pre and In order → $\Theta(n \log n)$

Step 4 post order traversal → $\Theta(n)$

overall complexity = $\Theta(n \log n)$

for each node log n is required
for n node n log n

Same for the post if post order traversal of BST is given then for BC and WC Time Complexity all thing will be same as pre order

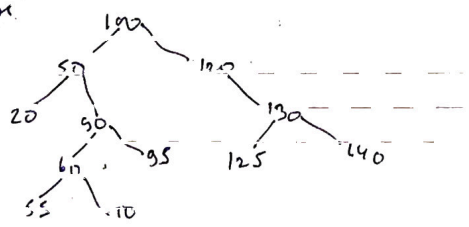
Can we reduce the complexity of n log n by $\Theta(n)$???

Yes

Suppose we have been given a
→ pre order Binary search tree

100, 50, 20, 90, 60, 55, 70, 95, 120, 130, 125, 140

traverse →



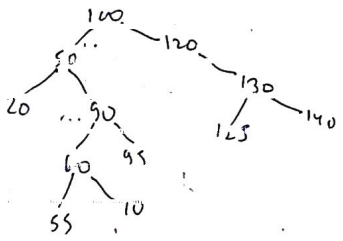
We have moved from left to right and each element is being compared exactly constant time so for n node $O(n)$

Now we can traverse in post order by $O(n)$

(1) [This was for if we have given pre BST then find post BST]

(2) [Now find if we have given post then find the pre BST]

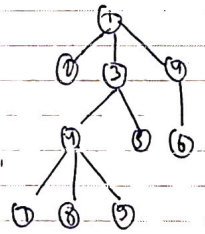
=) 20, 55, 70, 60, 95, 90, 80, 125, 140, 130, 120, 100



If min/max heap: In order traversal dis-gives final pre or post order traversal algo TC.

↳ nlogn

GateP98



Now here we have 2 child (final in order ??)

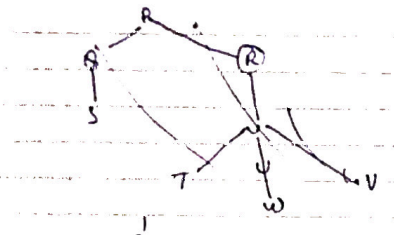
In in order we are confirm that (Left → Root → Right)

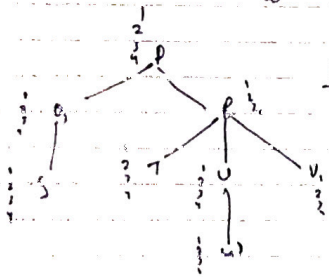
So (Left → Root → middle → Right

or (Left → middle → Root → Right

Solve it by both which one is option tick that.

Gate 14





Final Encoding

(1) Left — Root — Middle — Right

(2) Left — Middle — Root — Right

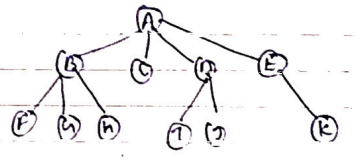
(1) Same is just like Recursion, (Left — Root — Middle — Right
1 2 3 4

QSPTRUWV)

(2) Left — middle — Root — Right
1 2 3 4

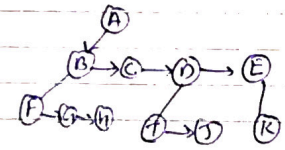
SQPTWURV

(यहाँ पर question में ये दिया गया है middle और Left या Right में कहां से कहां तक है)



To store this binary tree we need 4 pointer but we will try to implement it by binary tree by the help of two pointer

(1) Left most pointer
(2) Node Sibling

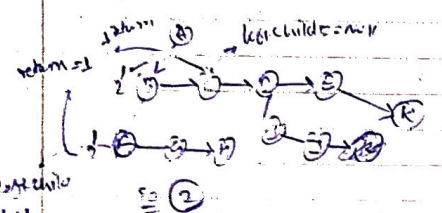


A dont have Sibling
B have Sibling C
C have Sibling D
Here K can be treated as left child

Linux

```
int val = 0
if (tree != NULL) {
    if (tree -> leftmostchild == NULL)
        value = 1
    Else
    1 value = DoSomething (tree -> leftmostchild)
    2 value = value + DoSomething (tree -> rightSibling)
    3
    return (value);
```

Solving this question by example



Eg ②

Binary search tree A 2

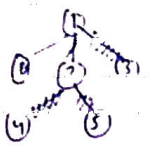
Nodes after the have not counted Growth
(D, E, K, I) this all due to left == Null

Counting trees

Unlabelled trees having n vertex

n^{n-2} formula

Eg



Vertex	Number of edges
3	1
4	2
5	3
2	1

8/ /

So the length of Prüfer code is 4
and length of Node are no of nodes in tree
∴ 6

so $\binom{n-2}{6-2}$: 4 · right

$n^{n-2} = 6^{6-2} = \boxed{6^4}$ labelled tree of n vertex

(n^{n-2}) is fact all tree node not for only binary

↳ not for Binary tree bcz for Binary tree
it should have 2 branch and at most 2 child

Verify $n^{(n-2)}$

$n=1$ $1^{-1} = 1$ ○

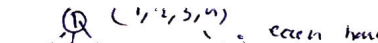
$n=2$ 2^{2-2} (1) ↔ (1)—(2) → (1) (2)

Dont see here this is left this is right here this all are equal

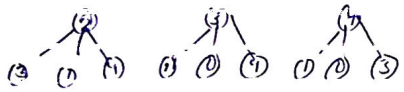
$n=3$ $3^{3-2} = 3$ (1)—(2)—(3)
(2)—(1)—(3)
(1)—(3)—(2)

$n=4 \quad n^{n-2} = 4^{4-2} = 4^2$

$= 16$ trees



each have tree. so total 4



and

$\frac{4!}{2!} = 12$ so $12+4 = 16$

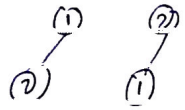
Mid it

all are same

Here many rooted labelled trees of n vertex

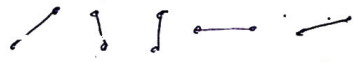
$n^{(n-2)} \times n = \boxed{(n^{n-1})}$ ans

n=2 $2^{2-1} = (2)$

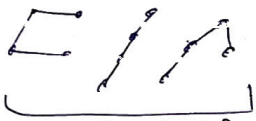


Cayley's tree is also known as minimum spanning tree

How many unlabelled trees of n vertices ?



this are one same always number



this all are same

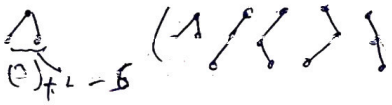
this is different

(remember your mean by sec degree)

How many labelled binary tree with n vertex

(here we will consider left and right child)

Node	No of tree
1	• (1)
2	(2)
3	(5)



How to calculate

internal node = 5 → Root node

left	right	
0	4	1 × 14 = 14
1	3	1 × 5 = 5
2	2	2 × 2 = 4
3	1	5 × 1 = 5
4	0	14 × 1 = 14
		42

by 4 node how many tree you can make that is 14

$$\frac{2n C_n}{n+1}$$

labelled Binary tree: $\left(\frac{2n C_n}{n+1}\right) n!$

unlabelled Binary tree: $\frac{2n C_n}{n+1}$

labelled tree: n^{n-2}

Rooted labelled tree: n^{n-1}

How many labelled

$$\left(\frac{2n C_n}{n+1} \times n!\right)$$

How many unlabelled Binary Search tree with n node

$$\left(\frac{2n C_n}{n+1}\right)$$

10/__/____

Gate 1998

A Complete n-array tree is one in which every node has either n sons. If x is the number of internal nodes of a complete n-array tree the no of leaves in it is given by

I	L
1	n
2	(n-1) + n
3	(n-2) + n + n.
4	(n-3) + n + n + n
⋮	
x	(n-(x-1)) + (x-1)n

$n-(x-1)+xn-x$

$xn - x + 1$

$x(n-1)+1$ So no of leaf node in n array Complete binary tree is

$L = I(n-1)+1$

I = Internal node

n array tree

Leaf Node

$I + L = n$

AVL tree

Why are we moving to AVL tree

(1) Binary tree : यहाँ पे जो भी element होता है वो left side में ही जाता है और greater होने पर right side में जाता है but binary tree में जब हम search करते हैं तो worst case में हमें search करने के लिए n steps लगते हैं

(2) Binary Search Tree : Here left side होगा smaller और right side होगा greater than parent node

When it is skewed Hence the worst time complexity = O(n)

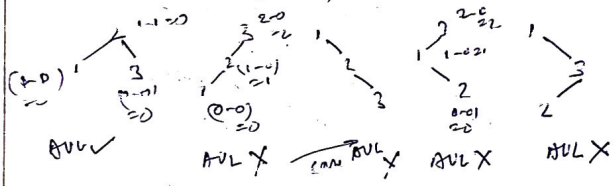
(3) AVL tree (height balanced tree)

$(H_L - H_R = 0, 1, -1)$

$$-1 \leq |H_L - H_R| \leq 1$$

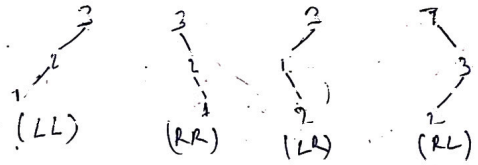
11/__/____

How many BST possible from 3 nodes having labels 1 to 3 (1, 2, 3)



4 type rotation

> Where the problem has been created from there we need to see

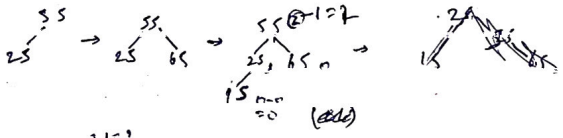


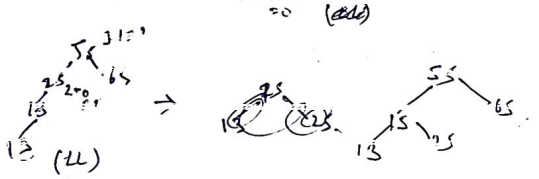
> LL → Right Rotation 1 time
> RR → Left Rotation 1 time
> LR → first left Rotation on left side of Node (n-1) pair and then Right Rotation
> RL → First Right Rotation on Right side of Node (n-1) pair and then Left Rotation

Note: कई Node जहाँ से insert हुआ है वहाँ से वहाँ से balance factor problem create कर रहा है वहाँ हम देखते कौनसा case है (LL, LR, RL, RR)

Construct an AVL Tree:

55, 25, 65, 15, 13, 20, 37, 51, 16, 53

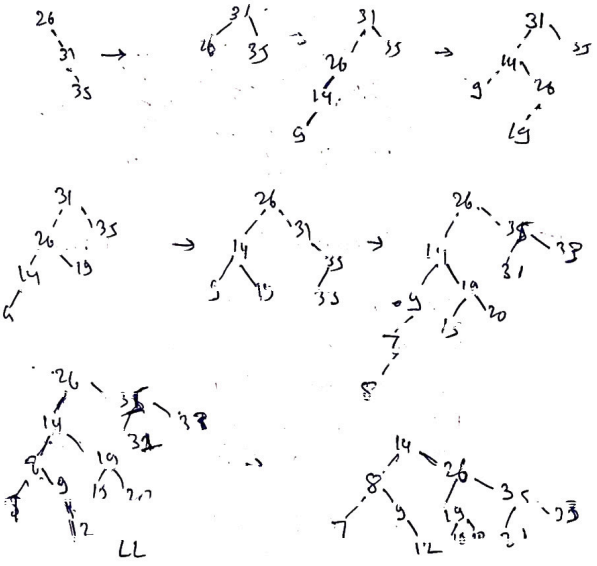




Same is still

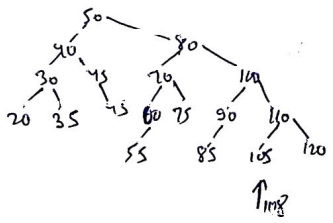
12/__/____

26, 31, 35, 14, 9, 19, 33, 20, 15, 7, 8, 17

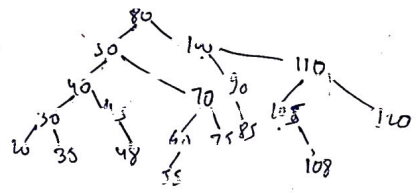


LL

Insert 108 in below AVL Tree



Balanced factor Prove problem at Node 50



Time Complexity for Insertion 1 node

Create node → O(1)
unsuccessful Search → log(n)
Insert → O(1)
Balance factor → log(n)
Rotation → O(1)

⇒ $O(\log n)$ → for 1 Node

If we are Constructing an AVL Tree Then TC

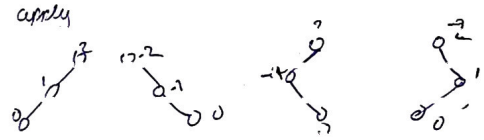
⇒ $O(n \log(n))$

13/__/____

BST	AVL TREE
if we have n nodes then maximum Rlogn nodes	if we have n nodes then maximum Rlogn) nodes
T.C = n^2	TC = nlogn

Rotation in AVL tree

PROBLEM :- Problem is that when you will delete the node then how to know which Rotation to apply



positive +ve

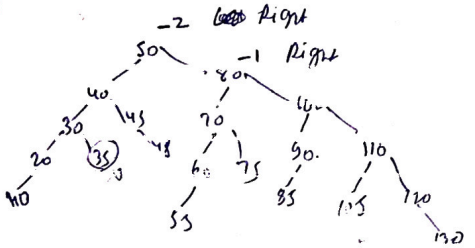
+ve +ve (Left – Left) Rotation
+ve -ve (Left – Right) Rotation
-ve +ve (Right – left) Rotation
-ve -ve (Right – Right) Rotation

Left +ve
Right -ve

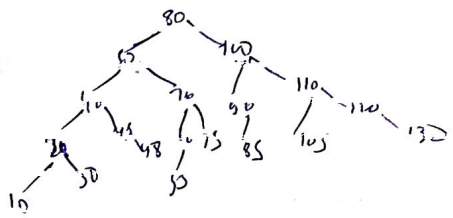
TC

Find AVL Tree after deleting 35 from below.

AVL Tree



Itself becomes 70 then it will create problem, we need to do 50 (Right Right Rotation)



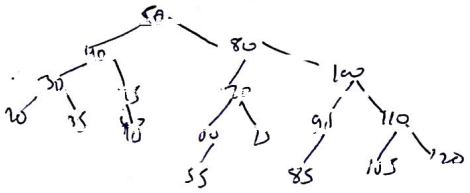
14/__/____

Deleting the Root Node:

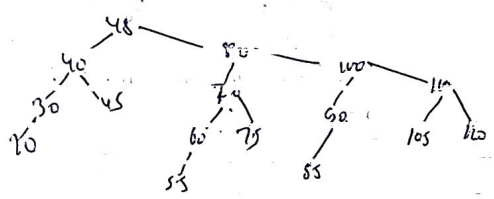
To delete the root node we do 2 process of BST predecessor and successor

(1) Predecessor (left then extreme Right)
(2) Successor (Right then extreme Left)

Delete 50



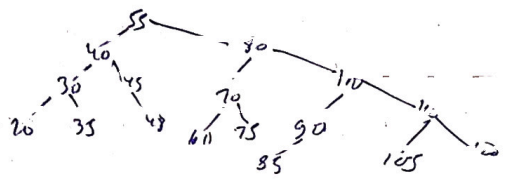
By predecessor (left then extreme Right = 48



By chance here no Rotation is Required

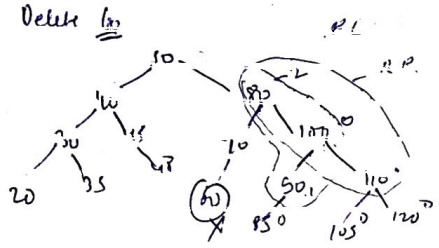
✗ Always we do (left - right) for balancefactor

Successor (Right - left , 55)

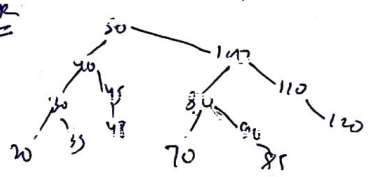


Good Question

Delete 50

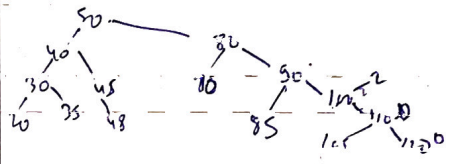


After deleting, the balance factor I have written here after (-2, 0). zero is coming so we can take '0' as +ve and -ve both and accordingly I have to match to option



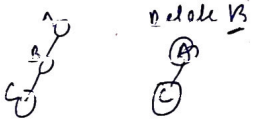
RL

First R then L



Here we have done RL Rotation but then also we have (Node) -2 (some value) which should not have in case so we do not need to do RL take RR

If we have single child node B



* Min and Max no of nodes in AVL tree of height h

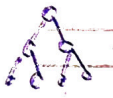
$\Rightarrow 2^0 + 2^1 + 2^2 \cdots 2^h$

$= 2^{h+1} - 1$

$n = 2^{h+1} - 1$ Max

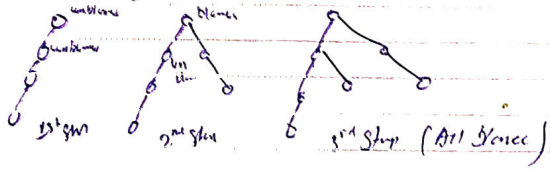
Min

Don't think its diagram will be



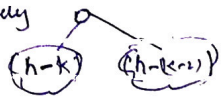
like this it can be in any shape only it should be balanced

for height 3 Example



GATE 2006

Mathematically



$(k = n-1)$

for height 5

(height - 4)

This will be solved by Recurrence Relation

$$N(h) = N(h-1) + 1 + N(h-2)$$

$$N(0) = 1$$
$$N(1) = 2$$

GATE 2009

10/1/

The binary tree T has n leaf node. The no. of nodes of degree 2 in T is

L = 2 | L = 3 | L = 4
2D = 1 | 2D = 2 | 2D = 3

means

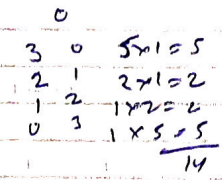
2 Degree Node + 1 = Leaf Node

Gate 2006

for a BST binary tree we know Standard

	Node	BST
K =	1	0
K =	2	
K =	3	

K = 4 =



3 0 5×1 = 5
2 1 2×1 = 2
1 2 1×2 = 2
0 3 1×5 = 5
14

means we are doing

Note 2006

$$T(n) = \sum_{K=1}^{n} T(k-1) * T(n-k)$$

Ques 20 #

In a binary tree for every node the difference b/w the no of nodes in the left and right subtree is at most 2. If the height of the tree $h > 0$ then the minimum no of nodes in tree is

=) (1) 2^{h-1} (2) $2^{h-1}+1$ (3) $2^{h}-1$ (4) 2^{h}

By option we can go with left skew

$N(h)$ min no of nodes in BST of height h which is $|N_L - N_R| \leq 2$

$$N(h) = \underbrace{N(h-1)}_{\text{Left}} + \underbrace{N(h-1)}_{\text{Right}} - 2 + 1$$

Node

Height same hoga left & right ka → 2 Node are subtracted from Null

$$\boxed{N(h) = 2N(h-1) - 1} \quad \boxed{N(1) \geq 2}$$

$= 2[2N(h-2)-1] - 1$

$= 2^2 N(h-2) - 2 - 1$

$= 2^2[2N(h-3)-1] - 2 - 1$

$= 2^3 N(h-3) - 2^2 - 2^1 - 2^0$

Binary tree means At most 2 child

17/__/__

$2^K N(h-K) - 2^{K-1} - 2^{K-2} - 2^{K-3} \ldots\ldots 2^0$

$2^K N(h-K) - [2^{K-1}]$

$K = h-1$

$2^K \cdot N(h-(h-1)) - 2^{h-1}$

$2^{h-1} N(h-K+1) - 2^{h-1} + 1$

$2^{h-1} \times 2 - 2^{h-1} + 1$

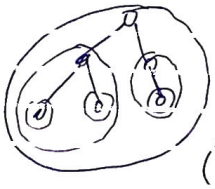
$2^h - 2^{h-1} + 1$

$2^{h-1} \times 2 - 2^{h-1} + 1$

$2^{h-1}[2-1] + 1$

$= \boxed{2^{h-1} + 1}$ Ans

Subtree



there are rounded one Subtree

ક્યાંક Node પ્રમાણે ક્યાંક દોરવા કે ગોળ કરવા Node ની right child કે so on consider ઓછામાં ઓછા even the leaf Node

2014

Consider a rooted n binary tree represented using pointers. The best upper bound on the time required to determine the number of subtree having exactly 4 nodes is $O(n^a \cdot \log^b n)$ find $(a+10b)$

Sol

best upper bound => finding worst case complexity

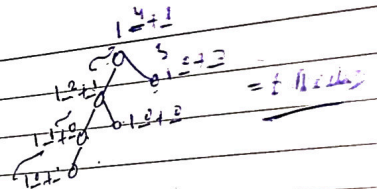
Subtree having exactly 4 nodes

So जहाँ भी 4 node मिलेगा increment by 1 और इसी तरह Count पे लगेगा कितने 4 Node वाले subtree है

To Count no of Node

```
ST(root) {
  if(root == NULL)
     return 0;
  else
    return 1+ ST(root->left) + ST(root->right);
```

(This will return total no of node)



```
Count = 0
ST (root) {
 if (root == NULL)
     return 0;
 else
    L = ST (root → left)
    R = ST (root → Right)
 if (1 + L + R == N)
     count ++;
 }
 return 1+L+R
```

Every node is visited exactly once and visiting need constant time
so total node is n so $\Theta(n)$

$n^a \log^b n$

$a=1, b=0$ so $a > 1 + 10 \times b$ = 1